DESIGN AND VERIFICATION OF A UART CONTROLLED MEMORY INTERFACE

Verilog RTL Project Report

MAIN IMPLEMENTATION

- UART Serial-to-Parallel and Parallel-to-Serial Interface
- 8 × 8 Receive and Transmit FIFOs
- Three-State Control FSM / Command Decoder
- 16 × 8 Dual-Port Memory with Circular Addressing

Prepared by: Muhammad Hammad

Project: UART Controlled Memory Interface

Language: Verilog

Date: 08 October 2026

TABLE OF CONTENTS

1. OVERALL SYSTEM ARCHITECTURE 3
2. CONTROL LOGIC FINITE-STATE MACHINE 4
3. MICRO-ARCHITECTURE 5
4. CONCLUSION 6

1. OVERALL SYSTEM ARCHITECTURE

The design implements a UART-controlled 16 × 8 memory interface using a serial receive path, receive FIFO, three-state control FSM, dual-port memory, transmit FIFO, and serial transmit path. The design intentionally omits baud-rate generation; one serial bit is transferred per system-clock cycle. Blue connections in the diagrams represent data or address buses, while red connections represent control and handshake signals.

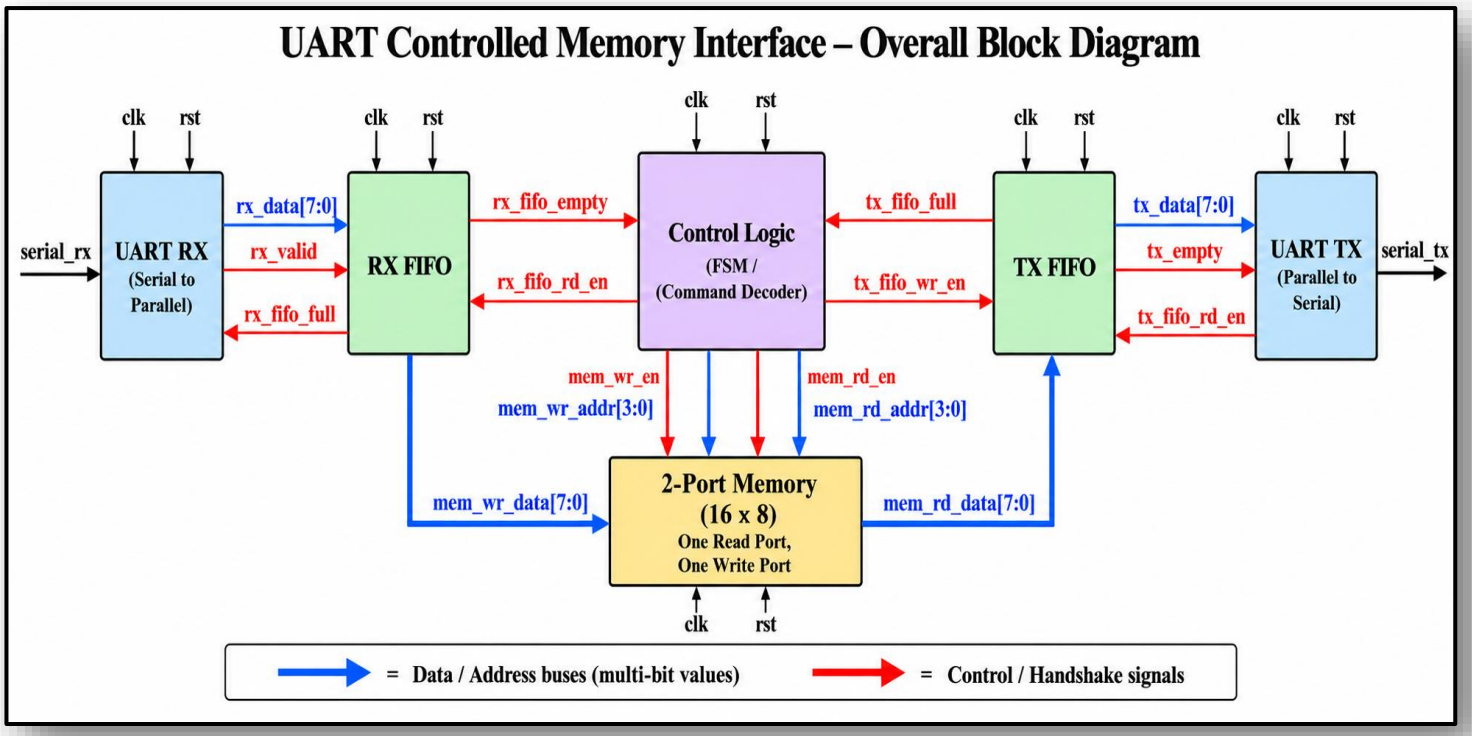


Figure 1. Overall block diagram of the UART controlled memory interface.

UART RX. Receives serial_rx, reconstructs each 8-bit byte, and asserts rx_valid when a complete byte is available. The byte is forwarded to the RX FIFO through rx_data[7:0].

RX FIFO. Buffers command and write-data bytes so that reception is independent of the controller processing rate. It reports rx_fifo_empty/full and supplies the memory write-data path directly.

Control Logic. Decodes the command fields, controls FIFO reads, generates the read/write enables, and maintains the current 4-bit memory address and remaining-byte count. It does not carry the payload data path.

2-Port Memory. Contains sixteen 8-bit locations with one write port and one read port. Write data arrives directly from the RX FIFO, while read data goes directly to the TX FIFO. Four-bit address arithmetic naturally wraps from 15 back to 0.

TX FIFO and UART TX. The TX FIFO buffers bytes returned by memory. UART TX removes the oldest byte when available and converts it from parallel form to serial_tx.

2. CONTROL LOGIC FINITE-STATE MACHINE

The controller uses only three states: WAIT_CMD, WRITE_DATA, and READ_DATA. This keeps the control path small because command decoding is performed when the command byte is at the head of the RX FIFO, while payload data moves directly between the FIFOs and memory.

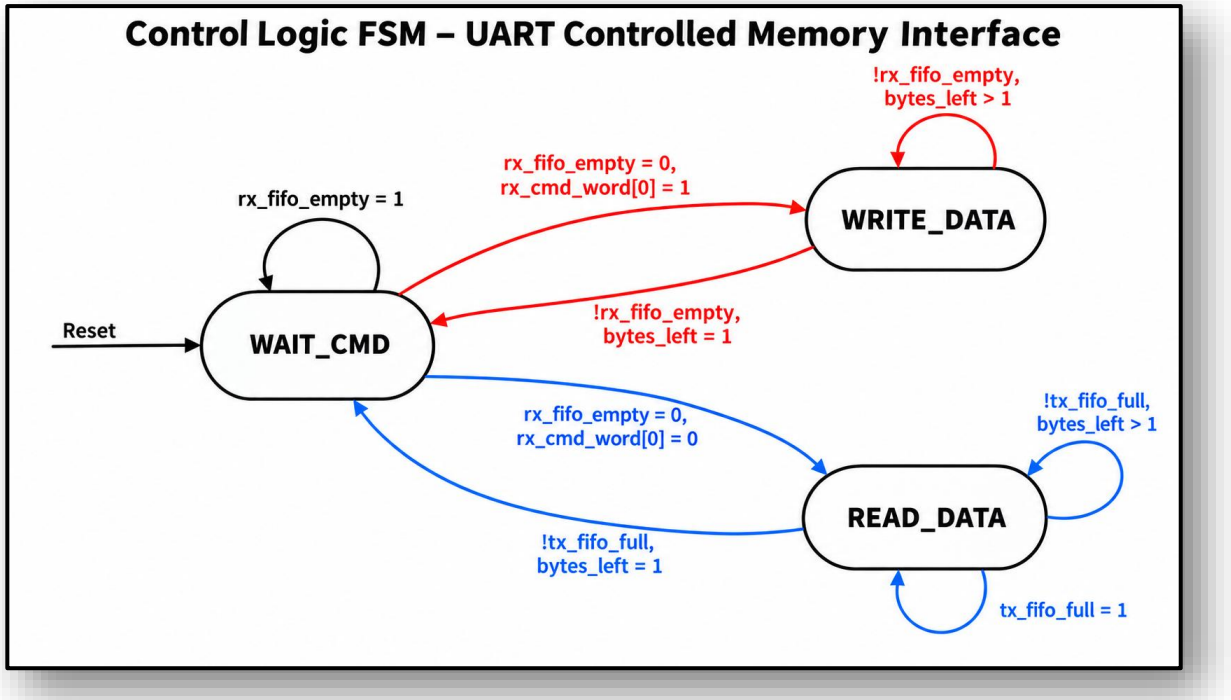


Figure 2. Three-state control FSM used by the controller.

WAIT_CMD. This is the reset and idle state. If rx_fifo_empty is high, the controller remains in WAIT_CMD. When a command byte is present, the controller reads it, loads the starting address from bits [4:1], forms the transfer count from bits [7:5] plus one, and examines bit [0]. A value of 1 selects WRITE_DATA; a value of 0 selects READ_DATA.

WRITE_DATA. The controller waits until the RX FIFO contains a payload byte. For each available byte it asserts rx_fifo_rd_en and mem_wr_en, presents current_addr on mem_wr_addr[3:0], decrements bytes_left, and advances the address. If bytes_left equals one, the final byte is written and the FSM returns to WAIT_CMD.

READ_DATA. The controller waits until the TX FIFO has space. It then asserts mem_rd_en and tx_fifo_wr_en while presenting current_addr on mem_rd_addr[3:0]. The memory byte is written directly into the TX FIFO. The address and count advance after each transfer, and the FSM returns to WAIT_CMD after the final byte.

The 4-bit current address register implements circular addressing automatically. For example, a four-byte transfer beginning at address 14 accesses locations 14, 15, 0, and 1. FIFO empty/full conditions therefore act as the flow-control conditions that pause the write or read sequence without introducing additional states.

3. MICRO-ARCHITECTURE

The micro-architecture expands the functional blocks into the registers, counters, storage arrays, and local control used in the Verilog implementation. The external behavior remains identical to the overall block diagram; the purpose of this view is to show how each module realizes that behavior internally.

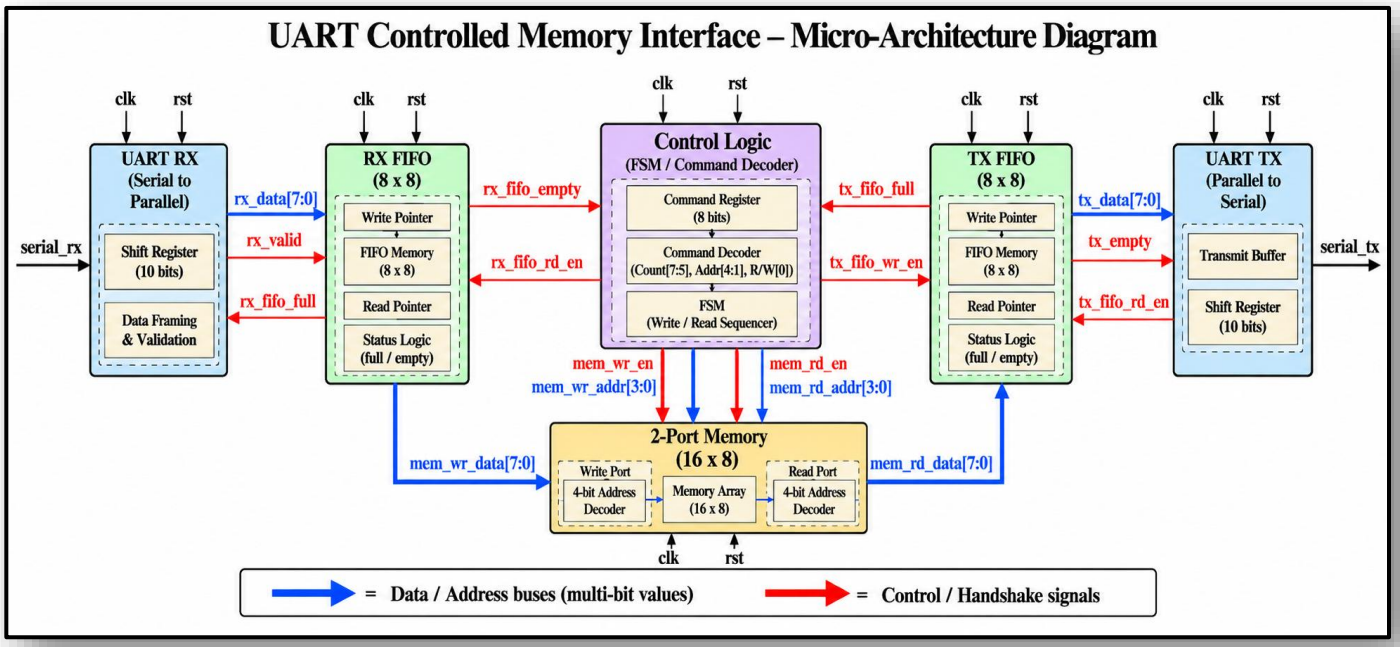


Figure 3. Micro-architecture of the UART controlled memory interface.

UART RX internals. The receiver uses a shift register and framing/validation logic to capture the start bit, eight LSB-first data bits, and stop bit. When a complete frame is accepted, rx_data is updated and rx_valid is pulsed for one clock.

FIFO internals. Both RX and TX FIFOs use an 8 × 8 storage array, 3-bit read and write pointers, and occupancy/status logic. The pointers wrap naturally after entry 7, while full and empty conditions are derived from the stored-byte count.

Controller internals. The command is interpreted as Count[7:5], Address[4:1], and W/R[0]. The FSM works with current_addr and bytes_left registers to sequence multi-byte operations and produce the memory and FIFO control signals.

Memory internals. The 16 × 8 array is accessed through separate read and write ports. The write side uses mem_wr_en and mem_wr_addr[3:0] with mem_wr_data[7:0] from the RX FIFO; the read side uses mem_rd_en and mem_rd_addr[3:0] and sends mem_rd_data[7:0] directly to the TX FIFO.

UART TX internals. The transmitter accepts an 8-bit FIFO byte, loads it into its transmit path, and serializes the byte with the required start and stop framing. The TX FIFO read enable prevents the next byte from being removed until the transmitter is ready.

Together, these blocks form a compact command-driven serial memory interface. The separation of data paths from the control path keeps the implementation easy to verify: the controller determines when and where a transfer occurs, while the payload bytes move directly through the FIFO-memory datapaths.

4. CONCLUSION

The project implements a modular UART-controlled memory interface in Verilog using a simple serial receive/transmit scheme, independent RX and TX FIFOs, a three-state controller, and a 16 × 8 dual-port memory. The design cleanly separates control from payload data movement, supports multi-byte read and write commands with automatic 4-bit address wrap-around, and uses FIFO full/empty conditions to regulate data flow without adding unnecessary controller states. Overall, the architecture remains compact, understandable, and suitable for directed verification of the required UART-to-memory functionality.